

El Gato | Tic Tac Toe - Enunciado

// 04 Estándares y buenas prácticas

04.1 Modelo de Madurez de Richardson – Nivel 2 requerido

0

The Swamp of POX

Un solo endpoint, todo via POST. No aplica.

1

Recursos

URLs distintas por recurso, pero un solo verbo. Insuficiente.

2

REQUERIDO

Verbos HTTP

Recursos + verbos correctos + códigos semánticos. Mínimo exigido.

3

BONUS

HATEOAS

Links de navegación en las respuestas. Optativo para nota máxima.

Aplicación práctica del Nivel 2

- **Verbos correctos:** GET para lectura, POST para crear y acciones de dominio, PUT/PATCH para actualizar si aplica.
- **Recursos nominales:** las URLs identifican sustantivos (`/games`, `/players`), nunca verbos (`/createGame`, `/doMove`).
- **Subrecursos para acciones:** `POST /games/{id}/join`, `POST /games/{id}/move` modelan acciones del dominio.
- **Códigos HTTP semánticos:** 201 al crear, 200 en operaciones exitosas, 400 en validaciones fallidas, 403 en acceso no autorizado, 404 si no existe, 409 en conflictos de estado, 422 cuando el estado del negocio no permite la acción, 502/503 en fallos de comunicación entre servicios.
- **Cuerpos de error estructurados:** nunca retornar solo el status HTTP; siempre incluir `error` (código de error) y `message` (descripción legible).

04.2 Códigos HTTP del proyecto

CÓDIGO	CUÁNDO USARLO EN ESTE PROYECTO
200 OK	GET exitoso, acciones (join, move) exitosas
201 Created	Nueva partida, nuevo jugador registrado
400 Bad Request	Movimiento inválido, posición fuera de rango, casilla ocupada, turno incorrecto, datos malformados
403 Forbidden	Jugador intenta consultar datos de otro jugador, o realizar acciones en partidas donde no participa
404 Not Found	Partida o jugador no existe con el ID dado
409 Conflict	Username duplicado, unirse a partida ya completa
422 Unprocessable Entity	Mover en partida no iniciada o ya finalizada
502 Bad Gateway	El server retornó un error 5xx al cliente
503 Service Unavailable	El server no respondió dentro del timeout configurado

04.3 Estructura de paquetes recomendada

GATO-SERVER

```
com.tuapp.gatoserver
├── controller
│   └── GameController
├── service
│   ├── GameService
│   ├── GameValidatorService
│   └── TimeoutService
├── model
│   ├── Game
│   ├── GameStatus (enum)
│   ├── GameType (enum)
│   └── PlayerSymbol (enum)
├── repository
│   └── GameRepository
├── dto
│   ├── CreateGameRequest
│   ├── MoveRequest
│   └── GameResponse
└── exception
    └── GameNotFoundException
```

GATO-CLIENT

```
com.tuapp.gatoclient
├── controller
│   ├── PlayerController
│   └── GameProxyController
├── service
│   ├── PlayerService
│   ├── GameRegistryService
│   └── ServerGatewayService
├── model
│   ├── Player
│   └── GameRegistry
├── repository
│   ├── PlayerRepository
│   └── GameRegistryRepository
├── dto
│   ├── CreateGameProxyRequest
│   ├── MoveProxyRequest
│   └── FallbackResponse
└── client
    └── GatoServerClient
```

```
└─ GameNotFoundException
└─ InvalidMoveException
└─ GlobalExceptionHandler
```

```
└─ GlobalServerClient
└─ exception
└─ ServerUnavailableException
└─ GlobalExceptionHandler
```

Buenas prácticas obligatorias

- Nunca retornar entidades JPA directamente: usar DTOs en todas las respuestas.
- Usar `@ControllerAdvice` con `@ExceptionHandler` para centralizar el manejo de errores.
- Las validaciones de reglas del juego viven en `GameValidatorService`, no en el controller.
- El cliente nunca hace `restTemplate.getForObject(...)` sin un bloque try-catch: toda llamada al server tiene fallback.
- Configurar timeout del `RestTemplate` o `WebClient` explícitamente en la configuración del cliente.
- Usar `@Transactional` en los métodos de servicio que modifican estado persistido.
- H2 habilitado en `application-dev.yml` con `spring.h2.console.enabled: true` — nunca en el perfil de producción.

← 03 MS Cliente

05 Extensión →

